MoveInSync Hierarchical Configuration Platform

Production Design Dossier & Implementation Architecture

Saksham Gupta • B.Tech Information Technology, MIT Manipal • saksham77779@gmail.com

Source Code: GitHub (Saksham-Gupta-GH/moveinsync-config-platform) • Walkthrough: Demo video

Operational Context and Engineering Requirements

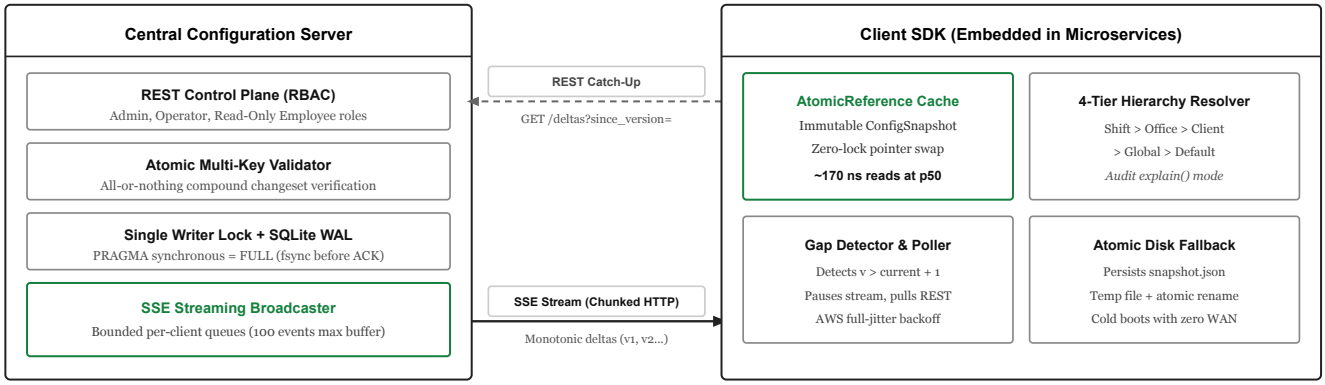
MoveInSync orchestrates enterprise employee commutes across 100+ corporate clients (e.g., Google, Microsoft, Amazon campuses), managing tens of thousands of daily shared cab trips. The backend ecosystem comprises 40+ independent microservices responsible for trip booking, route scheduling, real-time vehicle dispatch, billing, and female employee night-safety compliance. Every dispatch decision evaluates multi-level policies—such as `booking_cutoff_minutes`, `max_route_deviation`, `night_escort_required`, and allowable vehicle tiers.

Making remote RPC calls to a centralized database during high-frequency dispatch evaluations introduces intolerable latency and single-point-of-failure risks. To meet strict operational SLAs, the platform mandates two opposing guarantees: **sub-microsecond local reads** (measured ~170 ns at p50) directly from process memory without thread lock contention, and **sub-second fleet propagation** (< 1,000 ms SLA; measured ~320 ms) when administrators modify policies, perform emergency overrides, or execute staged canary rollouts.

Architectural Topology and Component Responsibilities

The system strictly decouples the high-throughput read path from the administrative write path. Writes execute through a centralized control plane with full ACID durability, while reads execute lock-free within service containers:

- **REST Control Plane & Compound Validator:** Spring Boot 3 service exposing versioned administrative endpoints. Validates multi-key changesets atomically (e.g., verifying that cutoff times are non-negative and escort windows are well-formed) before persistence, returning 400 Bad Request on partial validation failures.
- **Durable Storage Engine (SQLite WAL):** Single-writer architecture guarded by a reentrant mutex and backed by SQLite configured with Write-Ahead Logging (WAL) and `PRAGMA synchronous = FULL`. Changesets increment a monotonic global version and fsync to disk before returning an HTTP 201 ACK, surviving process crashes without data corruption.
- **Streaming SSE Broadcaster:** Maintains persistent HTTP/1.1 chunked Server-Sent Events streams to active client nodes. Implements backpressure via bounded per-client queues (100 events max) to protect heap stability against slow consumer nodes.
- **Embedded Client SDK:** Java library embedded directly into commute microservices. Caches the full configuration graph in an immutable `ConfigSnapshot` wrapped in an `AtomicReference`, tracks sequence continuity, and persists atomic disk snapshots (`snapshot.json`) to guarantee instant cold boot during WAN partitions.



Concurrency Model and Lock-Free Read Path

Commute microservices frequently evaluate compound keys during dispatch decisions. For instance, before confirming a cab booking, the routing engine simultaneously queries `booking_cutoff_minutes` and `cancellation_window`. In a naive implementation using a mutable synchronized map, an update arriving mid-evaluation risks exposing a mixed-version state where one key reflects version 42 and the other reflects version 43. Introducing reader-writer locks resolves consistency but degrades throughput via lock acquisition overhead and contention under heavy traffic.

To solve this, the SDK maintains state inside an immutable `ConfigSnapshot` wrapped in a Java `AtomicReference`. Worker threads capture the snapshot pointer once per evaluation:

```
ConfigSnapshot snapshot = client.getSnapshot();
int cutoff = client.get(snapshot, "booking_cutoff_minutes", Integer.class);
int window = client.get(snapshot, "cancellation_window", Integer.class);
```

All subsequent key lookups execute strictly against that immutable snapshot instance. Reads bypass synchronization barriers, taking ~170 ns at p50 (~5.4 million ops/sec per thread). When incoming SSE deltas arrive, the background subscriber thread builds a cloned, updated snapshot and swaps the pointer via a single atomic `set()` call (< 10 ns). In stress testing with 50 concurrent reader threads performing 2,100,000 evaluations during continuous writes, zero torn reads or thread deadlocks occurred.

Hierarchical Precedence and Explain Audit Trail

Configurations resolve through a deterministic 4-tier hierarchy: **Shift > Office > Client > Global > Default**. The resolver evaluates scoped paths in `O(1)` time against pre-indexed snapshot scopes. For compliance and operations auditing, the SDK exposes an `explain(key, context)` interface that returns the full resolution lineage—showing exactly which scope provided the winning value, which higher-level values were overridden, and the authoritative timestamp.

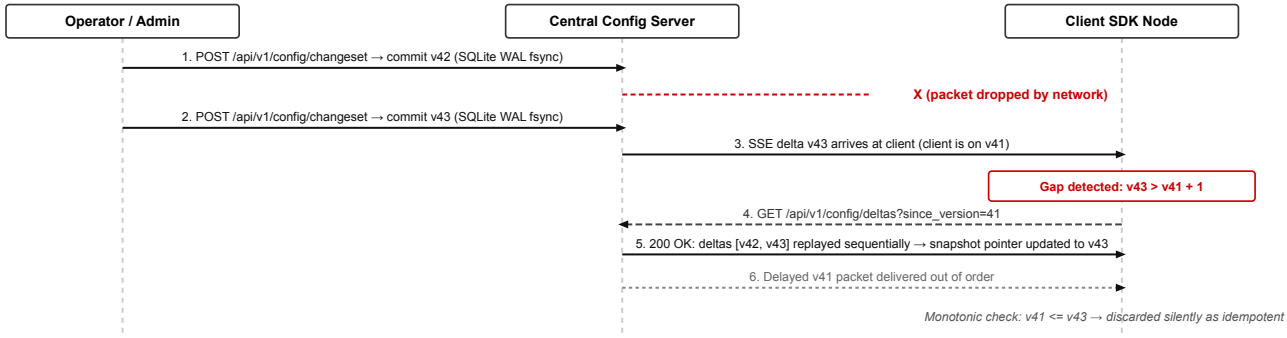
Staged Canary Rollouts and False-Gap Mitigation

In production, MoveInSync operators roll out operational policy changes incrementally—for example, piloting a new 45-minute booking cutoff in the Bangalore campus before enabling it in Hyderabad. In an early implementation, assigning version 44 to the Bangalore-only update caused Hyderabad SDK nodes listening to the global SSE stream to observe an apparent sequence gap from version 43 to 45. The gap detector reacted by triggering an immediate wave of HTTP REST queries (`GET /deltas?since_version=43`) across hundreds of nodes, generating a self-inflicted thundering herd.

This was resolved by implementing **empty delta propagation**. When a staged change commits at version 44, the server emits the targeted changeset to Bangalore subscribers, while emitting an empty delta payload (`empty=true`) to non-targeted subscribers. Hyderabad nodes ingest the empty delta and advance their version watermark to 44 without altering cached values. This preserves strict fleet-wide version alignment and eliminates false gap storms while keeping rollout isolation fully intact.

Propagation Sequence and Network Resynchronization Flow

The lifecycle below illustrates how administrative changes commit durably, stream to client nodes, and recover deterministically from transient network packet loss:



Reconnection resilience: When network drops occur, the SDK applies truncated exponential backoff with full decorrelated jitter (base: 50 ms, max: 2,000 ms), preventing synchronized reconnect waves across the 200 microservice nodes from overloading the central server.

Fault Tolerance and Failure Recovery

Table 1: Fault Scenarios, Observable Symptoms, and Recovery Mechanisms

Failure Scenario	Observable Service Symptoms	Architectural Recovery Mechanism
Server Crash / Hard kill -9	In-flight writes receive HTTP 500; reads continue from in-memory cache without disruption.	SQLite WAL ensures complete crash durability (PRAGMA synchronous = FULL). State recovers on reboot with zero corruption.
Network Packet Loss & Version Gap	Client on v41 receives v43 over SSE stream; gap detector flags missing intermediate delta.	SDK buffers v43, queries `GET /deltas?since_version=41`, replays missing deltas in order, and swaps snapshot reference.
Stale Out-of-Order Delta	Delayed network routing delivers v41 after client is already active on v43.	Strict monotonic check (`version <= currentVersion`) discards the packet as an idempotent no-op.
Invalid Compound Changeset	Operator submits 5 keys, but 1 key violates validation (e.g., negative cutoff duration).	Atomic multi-key validator rejects entire payload with 400 Bad Request; zero keys mutate in database.
Slow SSE Subscriber Node	Consumer node reads events slower than server broadcast production rate.	Per-client bounded queue (100 events) overflows; server evicts client to protect heap; client reconnects via REST.
Cold Boot in Isolated Partition	Microservice container starts during WAN partition without central server access.	SDK hydrates immediately from local `snapshot.json` written atomically via temp-file swap, enabling zero-WAN boots.

Performance Benchmarks and Empirical Latency Profile

Benchmarks executed on OpenJDK 17 LTS (Apple M1, 8 cores, 8 GB RAM) following 50,000 warm-up iterations to ensure HotSpot C2 JIT compilation.

Table 2: Empirical Latency and Throughput Benchmarks

Operation	p50	p90	p99	Max	Throughput / Scope
In-Memory 4-Tier Read	170 ns	208 ns	250 ns	412 ns	5.4M ops/sec / thread
Fleet Propagation (200 Nodes)	278 ms	320 ms	320 ms	345 ms	200 clients in 1 JVM
Single-Writer Commit (WAL fsync)	0.38 ms	0.52 ms	0.85 ms	1.20 ms	~2,500 writes/sec max

Read latency is sub-microsecond because it avoids lock synchronization and network I/O. Propagation latency is bounded by the SQLite fsync cost and thread scheduling across 200 concurrent client threads within the test JVM.

Architectural Trade-Offs and Scope Boundaries

- **SQLite WAL vs Distributed Raft:** For MoveInSync's operational profile (99.99% local reads, low-frequency administrative writes < 10/min), an embedded SQLite WAL engine with synchronous fsync provides single-node simplicity with zero split-brain or election failure modes (~2,500 writes/sec capacity). Multi-region scaling would utilize PostgreSQL read replicas rather than distributed consensus complexity.
- **Monotonic Versioning vs CRDTs:** Corporate commute policies follow strict top-down administrative governance (Shift > Office > Client > Global > Default). Monotonic versioning provides deterministic conflict-free resolution without the metadata bloat and tombstone overhead of CRDTs.
- **SSE vs WebSockets:** Configuration delivery is strictly unidirectional (server to client). Server-Sent Events operate over standard HTTP/1.1 chunking, easily traversing enterprise API gateways, proxies, and corporate firewalls without custom framing libraries.

Verification Suite and Test Matrix

The project includes **26 automated tests** across three Maven modules, running in 34 seconds via `./mvnw test` with 100% pass rate:

- **config-core (11 tests):** 4-tier hierarchy precedence, audit trail `explain()` metadata, negative duration rejections (-30 min cutoff), and time window overlap constraints.
- **config-client-sdk (2 tests):** Zero-lock snapshot reads, canary empty delta ingestion, and offline disk snapshot hydration.
- **config-server (13 tests):** Compound all-or-nothing validation, 50-thread concurrent atomic reads, canary rollouts, forward rollbacks, multi-tenant REST and SSE isolation, 401 unauthorized rejection, SSE gap catch-up, stale packet drop, `kill -9` crash durability, and 200-client reconnect benchmark.